

8/1/2026

ASSIGNMENT: 03

Transforming Raw Security Logs into
Structured, Reliable, and
Reproducible Data



Program	BS Artificial Intelligence
Internship	THE ARZENS Engineering Internship
Prepared By	Shawal Shamraiz
Program	BS Artificial Intelligence
Track	AI, Automation & Security Engineering
Project Type	AI + Automation + Cybersecurity Engineering (Hybrid)
Version	Version 1.0 — Proposal Draft
Date	1st August 2026

TASK 1

Raw vs. Normalized Data in Security Log Analysis

Log data doesn't really come in one shape. A Windows server logs a failed login as event code 4625; a Linux box just dumps "authentication failure" into syslog. A firewall doesn't even log a username , just an IP and a port. Timestamps are their own headache: some devices stamp in UTC, some in local time with no documented offset, and a few older appliances skip the year entirely. Field names drift too , "src_ip" here, "source_address" there, sometimes just "IP." Add duplicate entries from redundant forwarders, plus firewalls, IDS/IPS boxes, web servers, and random SaaS apps all built by different teams, and raw logs stop being searchable across the board. It just turns into noise stacked on more noise.

Normalization fixes this by forcing everything into one shape , one field for source IP, one for username, one shared timestamp format. That's basically why SIEM platforms exist: they pull logs from a dozen unrelated products and parse them into something a correlation rule can compare. With consistent field names and formats, machine learning models can pick up on patterns and anomalies without heavy preprocessing, since they're not fighting a different structure per source. It also makes dashboards and reports more useful — an analyst can glance at a visualization and spot a trend or a suspicious spike instead of digging through raw text.

But normalization has a cost most people don't think about until it bites them. Basically it's compression, and compression throws something out. Push it too hard and you lose the Windows logon type, the exact TLS cipher, the process command line — details that don't fit the schema but matter during an actual investigation. Over-normalize and you get false uniformity, where genuinely different events start looking identical, raising the odds an analyst draws the wrong conclusion mid-investigation. Logs should get easier to read, not thinner on evidence.

Say ten systems — a couple domain controllers, a VPN gateway, some web apps — all log failed logins within the same hour, each stamped differently. Alone, none of it looks like much. Normalize the timestamps and usernames into one timeline, though, and the same accounts failing across all ten systems within minutes jumps out immediately — a credential stuffing attack, visible only once correlation makes it visible.

Stop normalizing the moment it starts deciding, quietly, which questions you're allowed to ask later.